Lab 02

TITLE

“Hands-On Git and Version Control”

Name:

Mehmoona Abid

Registration number:

(2023-BSCS-97)

Section:

A

GitHub username

mehmoonaabid

Lab 2: Hands-On Git and Version Control

Estimated duration: 3 hours

Submission repository: DevOps

Submission directory: Labs/Lab02

Screenshot directory: Labs/Lab02/screenshots

Complete the practical work in the repository required by each task. Submit all evidence to your GitHub repository named exactly DevOps using the directory structure shown below.

```
DevOps/
└── Labs/
    └── Lab02/
        ├── README.md
        ├── notes.txt
        ├── main.py
        ├── Lab2_Solution.pdf
        └── screenshots/
            ├── git_installation.png
            ├── repo_private.png
            └── ...
```

Navigation: Click any task below to jump to its instructions.

Task List

Getting Started

Task 1: Create Private GitHub Repository

Task 2: Connect Repository via SSH

Task 3: Configure Git Username and Email

Task 4: Explore the .git Folder

Task 5: Local Repository Management

Task 6: File Status & Staging

Task 7: Branch Creation Using GitHub GUI

Task 8: Branch Creation and Push Using Git Bash

Task 9: Branching & Merging

Task 10: Pull Request and Branch Review (GitHub GUI)

Task 11: Detailed Branch Strategy (Develop/Staging)

Task 12: Code Review Workflow

Task 13: Branch Cleanup Best Practices

Bonus Task: Simulated Team Collaboration

Exam Evaluation Questions

Mandatory Screenshot and Autograding Rules

Required Automated-Grading Filenames

The current Lab 2 grader checks these 37 files:

git_installation.png

repo_private.png

ssh_keygen.png

github_sshkey.png

ssh_clone.png

git_identity.png

git_config_list.png

git_folder.png

delete_git.png

git_init.png

first_commit.png

first_push.png

status1.png

commit_notes.png

bugfix_branch_gui.png

bugfix_branch_local.png

feature_db_branch.png

branch_create.png

feature_commit.png

merge.png

push_branches.png

pr_creation.png

pr_merge.png

branch_delete.png

branch_strategy.png

branch_merges.png

final_merge.png

pr_create_details.png

pr_assigned_reviewer.png

pr_request_changes.png

pr_rejected.png

pr_updated_with_commits.png

pr_merge_confirm.png

pr_merged.png

pr_branch_deleted.png

remote_branch_deleted.png

remote_branch_delete_cmd.png

Each of these 37 checks has equal weight. Bonus and additional exam-practice screenshots may be reviewed separately and are not included in these 37 automated checks.

Getting Started

1. Install Git on your PC.

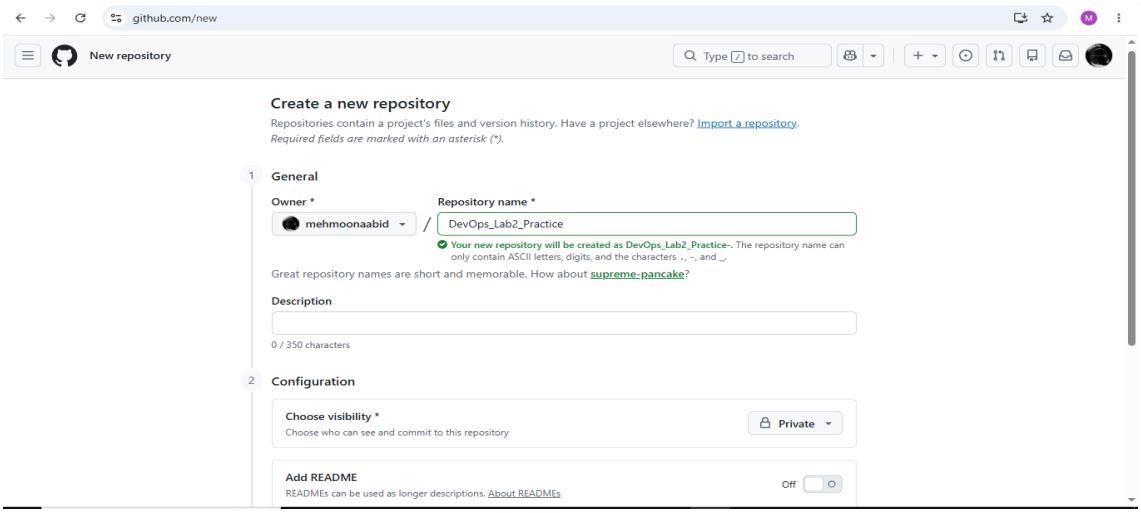
- Download Git for Windows, Mac, or Linux
- Save the evidence as Labs/Lab02/screenshots/git_installation.png.

```
Last login: Wed Oct 14 09:31:18 2026 from 192.168.231.1
mehmoonaabid@ubuntu:~$ sudo apt update
[sudo: authenticate] Password:
Hit:1 http://security.ubuntu.com/ubuntu resolute-security InRelease
Hit:2 http://pk.archive.ubuntu.com/ubuntu resolute InRelease
Hit:3 http://pk.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:4 http://pk.archive.ubuntu.com/ubuntu resolute-backports InRelease
24 packages can be upgraded. Run 'apt list --upgradable' to see them.
mehmoonaabid@ubuntu:~$ sudo apt install git openssh-client -y
git is already the newest version (1:2.53.0-1ubuntu1).
git set to manually installed.
openssh-client is already the newest version (1:10.2p1-2ubuntu3.7).
openssh-client set to manually installed.
The following packages were automatically installed and are no longer required:
  linux-headers-7.0.0-30          linux-image-unsigned-7.0.0-30-generic    linux-modules-7.0.0-30-generic  linux-tool
  linux-headers-7.0.0-30-generic  linux-main-modules-zfs-7.0.0-30-generic  linux-tools-7.0.0-30
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 24
mehmoonaabid@ubuntu:~$ git --version
git version 2.53.0
mehmoonaabid@ubuntu:~$
```

Task 1: Create Private GitHub Repository

1. **Create a new private repository named exactly DevOps_Lab2_Practice on GitHub.**
2. Take a split-screen screenshot showing the repository is private and your required terminal prompt. Save it as Labs/Lab02/screenshots/repo_private.png.



Task 2: Connect Repository via SSH

1. Generate a new SSH key

using PowerShell: *ssh-keygen -t*

ed25519 -C

"your_email@example.com" cat

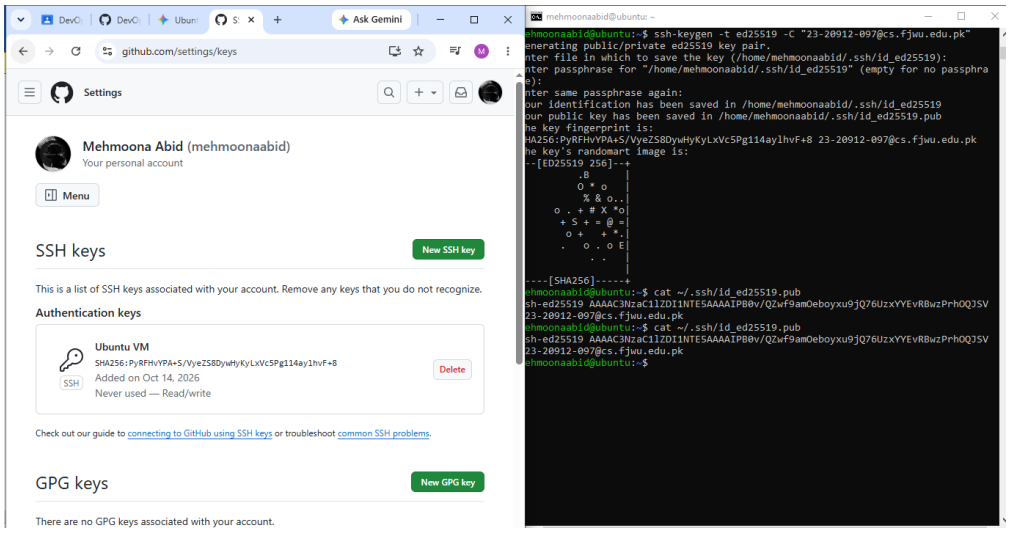
~/.ssh/id_ed25519.pub

- Save it as Labs/Lab02/screenshots/ssh_keygen.png. Do not show the private key contents.

```
mehmoonaabid@ubuntu:~$ git --version
git version 2.53.0
mehmoonaabid@ubuntu:~$ ssh-keygen -t ed25519 -C "23-20912-097@cs.fjwu.edu.pk"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/mehmoonaabid/.ssh/id_ed25519):
Enter passphrase for "/home/mehmoonaabid/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Passphrases do not match.  Try again.
Enter passphrase for "/home/mehmoonaabid/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/mehmoonaabid/.ssh/id_ed25519
Your public key has been saved in /home/mehmoonaabid/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:+fQCooImtGmHpTAfwsVR19RweDsW85xHUrhUJCbN07E 23-20912-097@cs.fjwu.edu.pk
The key's randomart image is:
+--[ED25519 256]--+
|    ... .oo+ooo*=.|
|   . . .  o.=oB.o.|
|    o      . B *E |
|. .       . + = . |
|+o..   . S o . .  |
|o=*. . . + .      |
|oBo..     o .     |
|+ ..       .      |
|                  |
+----[SHA256]-----+
mehmoonaabid@ubuntu:~$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIOOlm5XtJjym85isOel7XJyHRGk4p1iIThbGmVANfvFc 23-20912-097@cs.fjwu.edu.pk
mehmoonaabid@ubuntu:~$
```

2. Add your SSH public key to GitHub (Settings > SSH and GPG keys).

- Save a split-screen screenshot as Labs/Lab02/screenshots/github_sshkey.png.



3. Clone your DevOps_Lab2_Practice repository using SSH.

git clone

git@github.com:<yourusername>/DevOps_L

ab2_Practice.git • Save it as

Labs/Lab02/screenshots/ssh_clone.png.

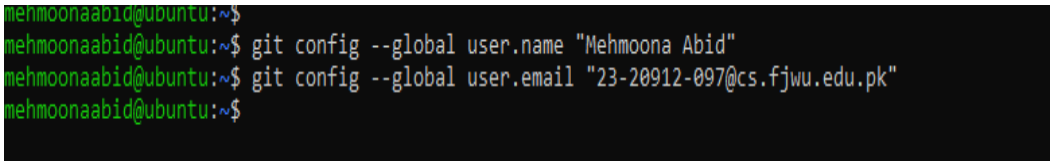


Task 3: Configure Git Username and Email

1. **Set up your Git identity (this ensures all commits are linked to you):**
2. git config --global user.name "Your Name"

git config --global user.email "your_email@example.com"

- Save a screenshot of your terminal as git_identity.png.



2. Verify your configuration:

git config --list

• Screenshot as git_config_list.png.

```
mehmoonaabid@ubuntu:~$ git config --list
user.name=Mehmoona Abid
user.email=23-20912-097@cs.fjwu.edu.pk
mehmoonaabid@ubuntu:~$
```

Task 4: Explore the .git Folder

1. Navigate into your cloned repository folder.
2. Show hidden files and locate the .git directory.
3. Explore what's inside using: *ls -a .git*

• Take a screenshot as git_folder.png.

```
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ pwd
/home/mehmoonaabid/DevOps_Lab2_Practice
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ cd ~/DevOps_Lab2_Practice
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ ls -a
.  ..  .git
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ ls -a .git
.  ..  HEAD  config  description  hooks  info  objects  refs
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$
```

4. Note: This folder contains history, logs, and config for your repository. Do not modify these files manually.

Task 5: Local Repository Management

1. Delete the existing .git folder from your cloned repo using Git Bash:

rm -rf .git git status ==> error why?

• Take a screenshot as delete_git.png.

```
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ cd ~/DevOps_Lab2_Practice
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ pwd
/home/mehmoonaabid/DevOps_Lab2_Practice
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ rm -rf .git
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git status
fatal: not a git repository (or any of the parent directories): .git
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$
```

2. Re-initialize the local git repository:

git init

Save a screenshot as git_init.png.

```
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /home/mehmoonaabid/DevOps_Lab2_Practice/.git/
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$
```

3. Add a file named README.md and commit it:

echo "# Lab02 Git Practice" >> README.md

git add README.md

git commit -m "Initial commit"

Save a screenshot as first_commit.png.

```
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ echo "# Lab02 Git Practice" >> README.md
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git add README.md
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Initial commit"
[master (root-commit) 5d2aaca] Initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$
```

4. Connect your local repo to GitHub and push:

git remote add origin
git@github.com:<yourusername>/DevOps_Lab2_Practice.git

git branch -M main

git push -u origin main

Save a screenshot as first_push.png.

```
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git remote add origin git@github.com:mehmoonaabid/DevOps_Lab2_Practice.git
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git branch -M main
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 244 bytes | 61.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:mehmoonaabid/DevOps_Lab2_Practice.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$
```

Task 6: File Status & Staging

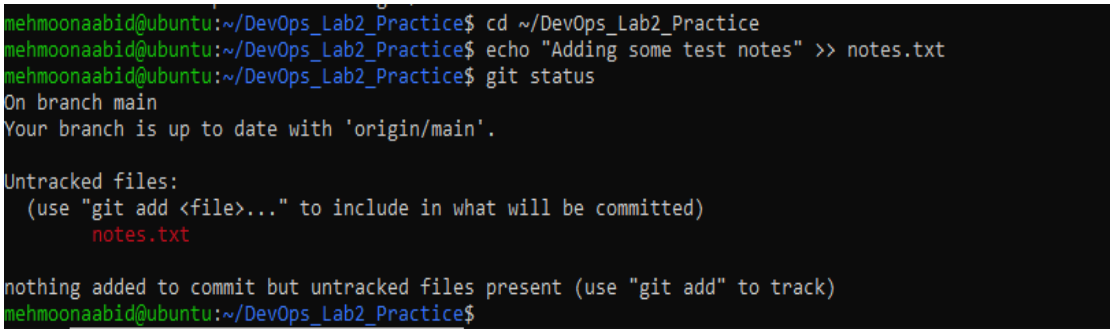
1. Create a new file notes.txt and write a note.

echo "Adding some test notes" >> notes.txt

2. Check status:

git status

- Screenshot as status1.png.

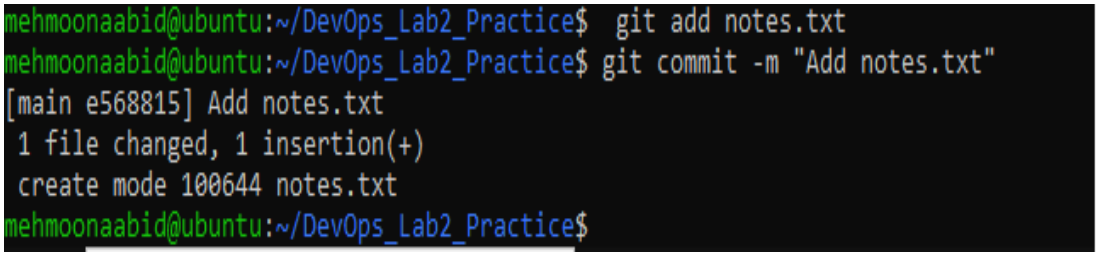


3. Stage and commit:

git add notes.txt

git commit -m "Add notes.txt"

- Screenshot as commit_notes.png.



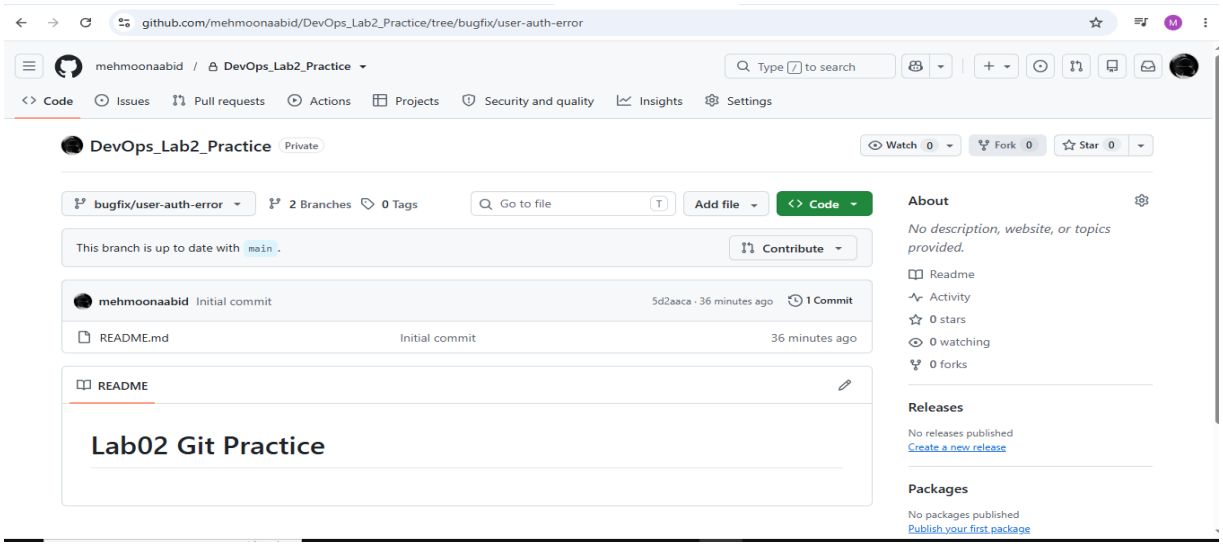
Edit notes.txt and repeat status/commit steps.

Task 7: Branch Creation Using GitHub GUI

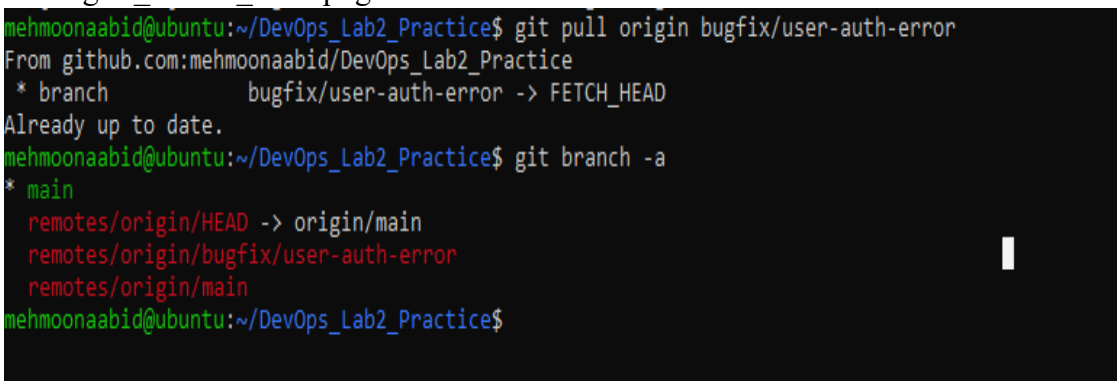
1. On GitHub (web interface), create a branch named bugfix/user-auth-error.
2. Pull the branch to your local repository to sync.

git pull origin bugfix/user-auth-error

- Take a screenshot of branch creation in the GUI as bugfix_branch_gui.png.



- Take a screenshot of your local git branch list after pulling as bugfix_branch_local.png.



Task 8: Branch Creation and Push Using Git Bash

1. Create a branch named feature/db-connection using Git Bash:

git checkout -b feature/db-connection

2. Push the branch to the remote repository:

git push origin feature/db-connection

• Take a screenshot of the branch creation and push as feature_db_branch.png.

```
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ cd ~/DevOps_Lab2_Practice
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git checkout -b feature/db-connection
Switched to a new branch 'feature/db-connection'
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git push origin feature/db-connection
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 306 bytes | 25.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/db-connection' on GitHub by visiting:
remote:      https://github.com/mehmoonaabid/DevOps_Lab2_Practice/pull/new/feature/db-connection
remote:
To github.com:mehmoonaabid/DevOps_Lab2_Practice.git
 * [new branch]      feature/db-connection -> feature/db-connection
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$
```

Task 9: Branching & Merging

1. Create and switch to a branch feature-1:

git checkout -b feature-1

• Screenshot as branch_create.png.

```
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$  cd ~/DevOps_Lab2_Practice
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git status
On branch feature/db-connection
nothing to commit, working tree clean
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git branch
  feature/db-connection
* main
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git checkout -b feature-1
Switched to a new branch 'feature-1'
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$
```

2. Modify main.py (add a function) and commit.

git add main.py

git commit -m "Add

new function to

main.py" •

Screenshot as

feature_commit.pn

g.

```
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ nano main.py
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git add main.py
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Add new function to main.py"
[feature-1 d809dfc] Add new function to main.py
 1 file changed, 4 insertions(+)
 create mode 100644 main.py
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$
```

3. Switch back to main and merge:

git checkout main git merge feature-1

• Screenshot as merge.png.

```
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git merge feature-1
Updating e568815..d809dfc
Fast-forward
 main.py | 4 ++++
 1 file changed, 4 insertions(+)
 create mode 100644 main.py
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$
```

Merging directly into main from your local terminal is widely considered a risky anti-pattern, especially on a team. In professional software development, using a Pull Request (PR) or Merge Request (MR) is the gold standard.

4. Push all branches: git push origin main

git push origin feature-1

• Screenshot as push_branches.png.

```
 create mode 100644 main.py
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 388 bytes | 35.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:mehmoonaabid/DevOps_Lab2_Practice.git
   5d2aaca..d809dfc  main -> main
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git push origin feature-1
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature-1' on GitHub by visiting:
remote:      https://github.com/mehmoonaabid/DevOps_Lab2_Practice/pull/new/feature-1
remote:
To github.com:mehmoonaabid/DevOps_Lab2_Practice.git
 * [new branch]      feature-1 -> feature-1
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$
```

Task 10: Pull Request and Branch Review (GitHub GUI)

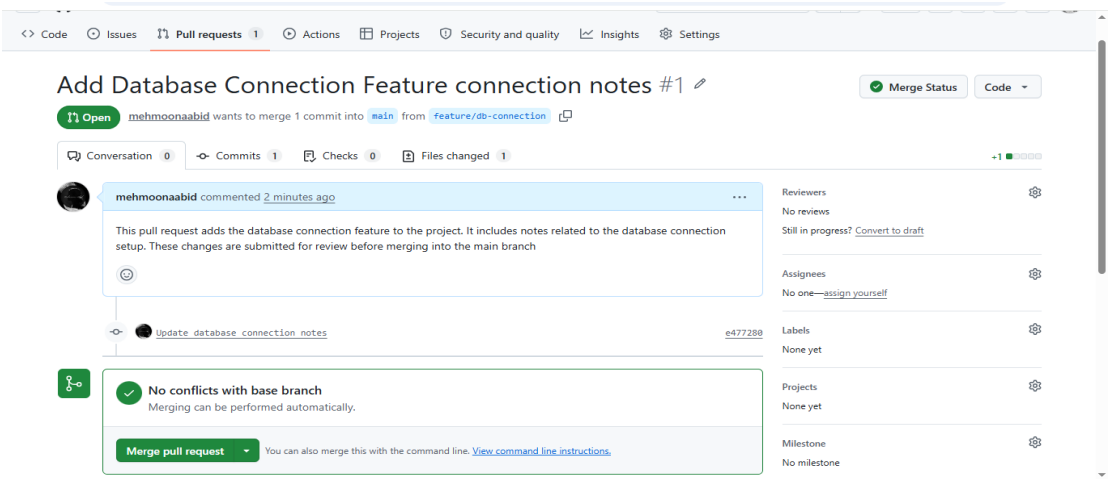
On GitHub, create a Pull Request from the branch feature/db-connection to main.

Review the Pull Request and merge it using the GitHub GUI.

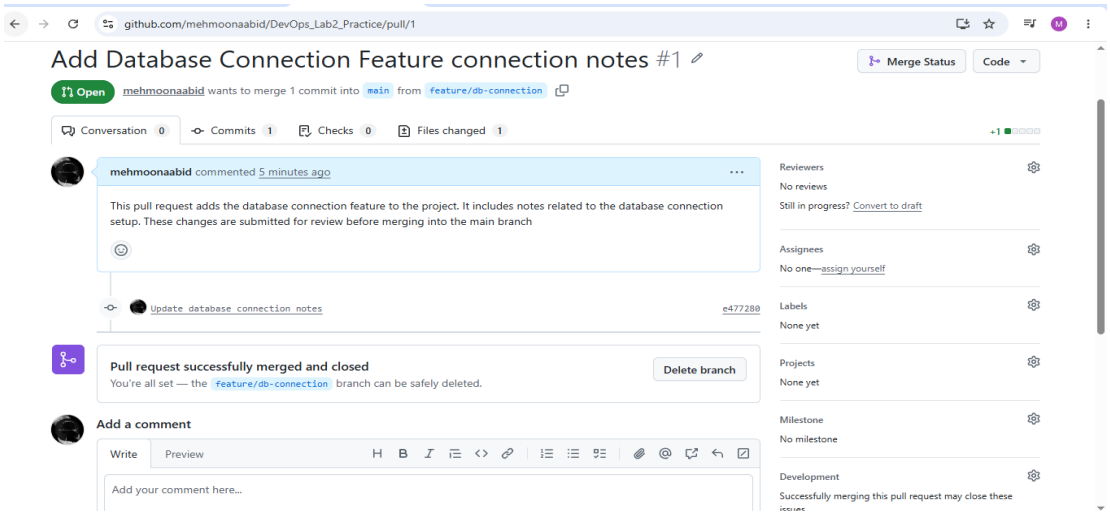
After merging, delete the feature/db-connection branch using the GitHub GUI.

Take a screenshot of:

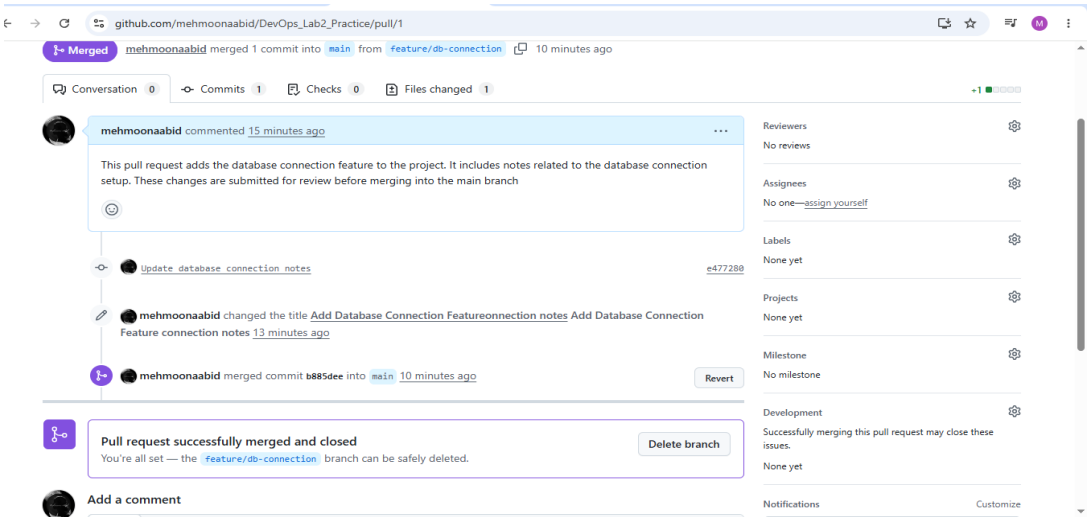
The Pull Request creation (pr_creation.png)



The Pull Request review and merge (pr_merge.png)



The branch deletion (branch_delete.png)



Task 11: Detailed Branch Strategy (Develop/Staging)

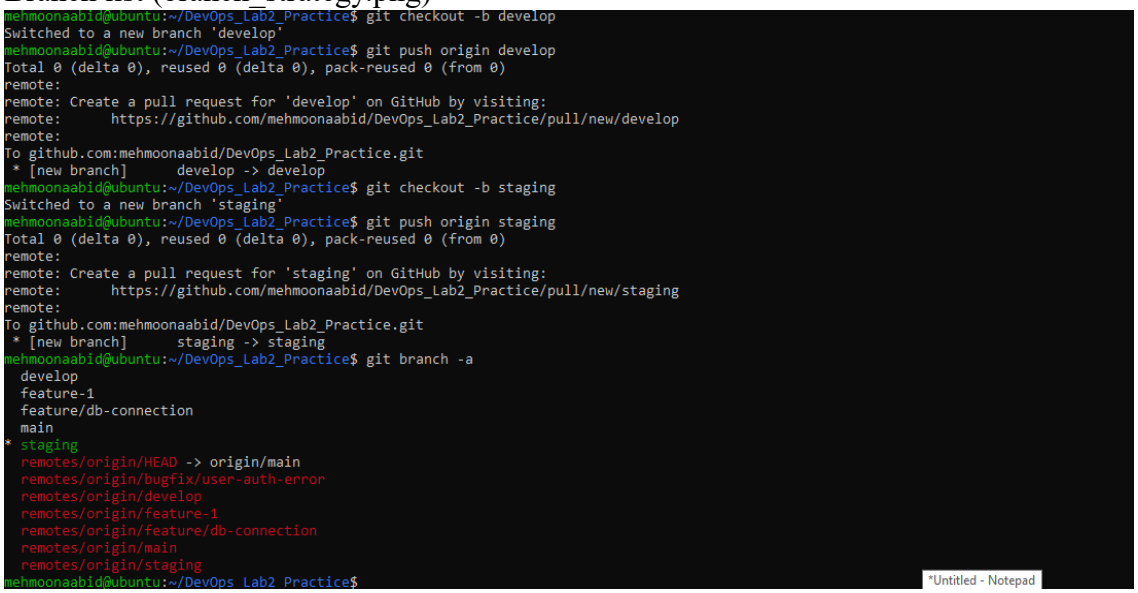
1. Create the following branches to simulate a professional branching strategy:
   - develop
   - staging
   - feature/*
   - bugfix/*
2. Example workflow:

- Developers work on feature/* and bugfix/* branches.

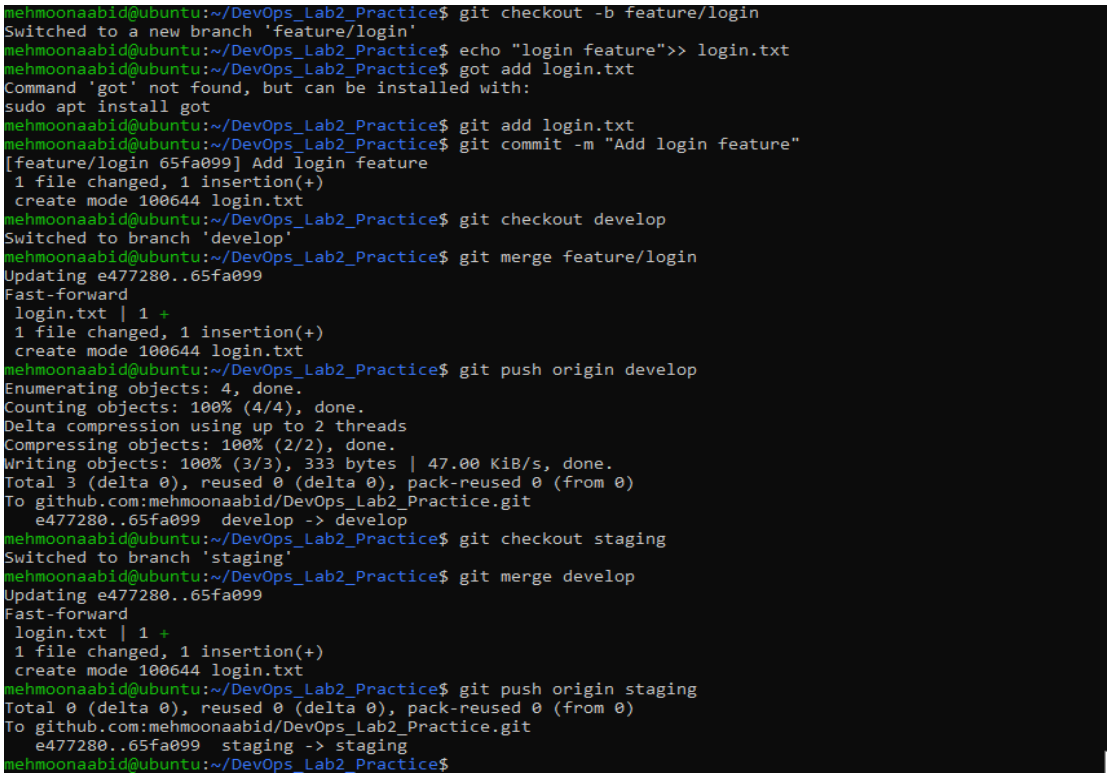
- Merge into develop after completion.
- develop is merged into staging for testing.
- Finally, staging is merged into main (production).

3. Document the workflow with screenshots:

- Branch list (branch_strategy.png)



- Merges into develop and staging (branch_merges.png)



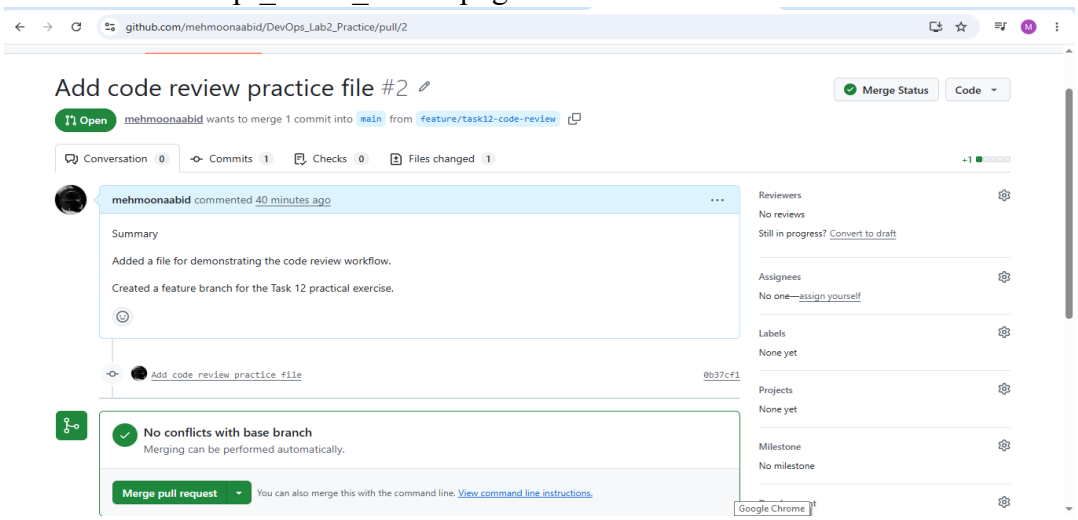
- Final merge into main (final_merge.png)

```
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git push origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 634 bytes | 42.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:mehmoonaabid/DevOps_Lab2_Practice.git
   b885dee..e245143  main -> main
```

Task 12: Code Review Workflow

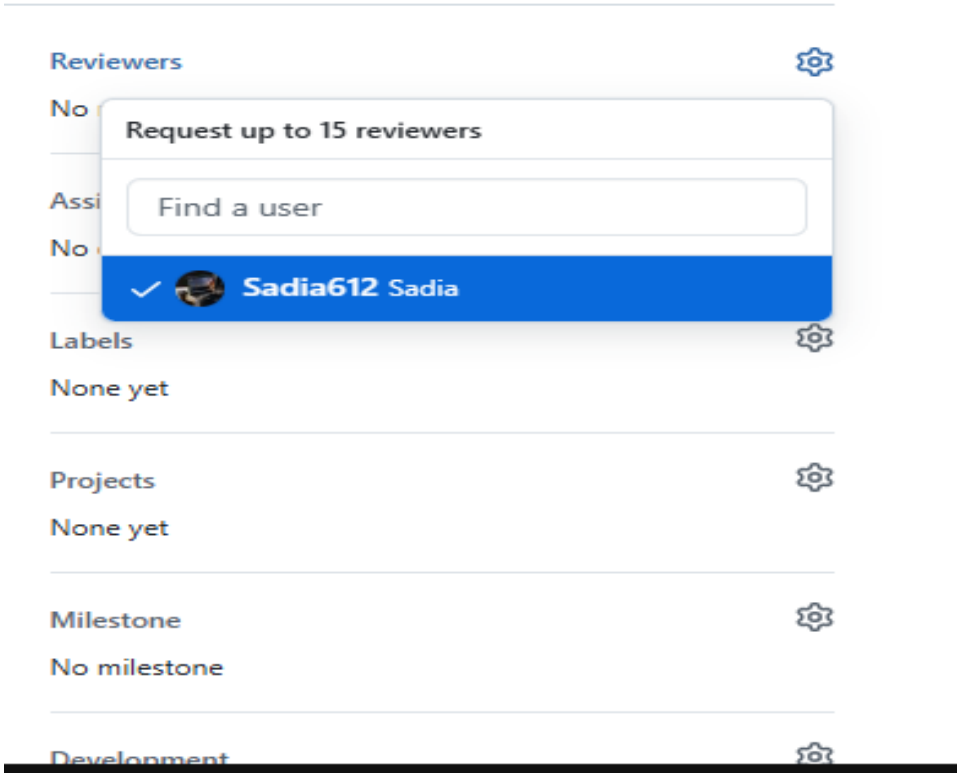
1. Create a Pull Request (PR) / Merge Request (MR):

- From a feature branch into main.
- Add a clear title and description summarizing the changes.
- Screenshot as pr_create_details.png.



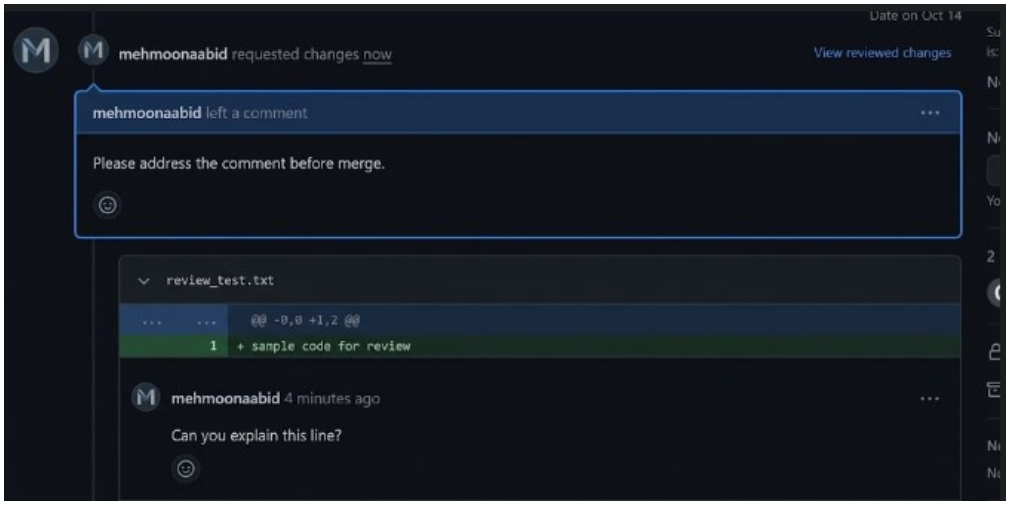
2. Assign a reviewer (teammate or second account).

- Screenshot as pr_assigned_reviewer.png.



3. Reviewer Actions (capture screenshots for each):

- Request changes and add comments → pr_request_changes.png

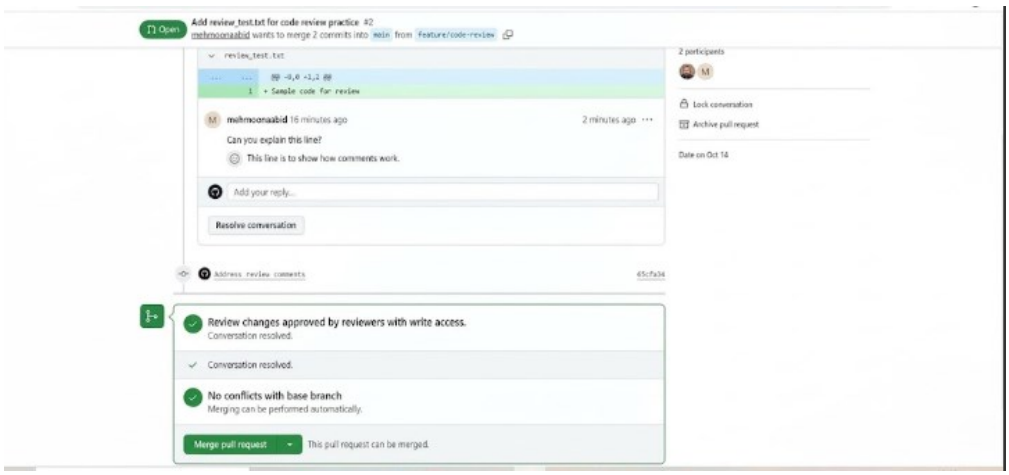


- Reject or close a practice PR → pr_rejected.png. Use a separate practice PR if necessary so all required review states can be demonstrated safely.

4. Author Actions:

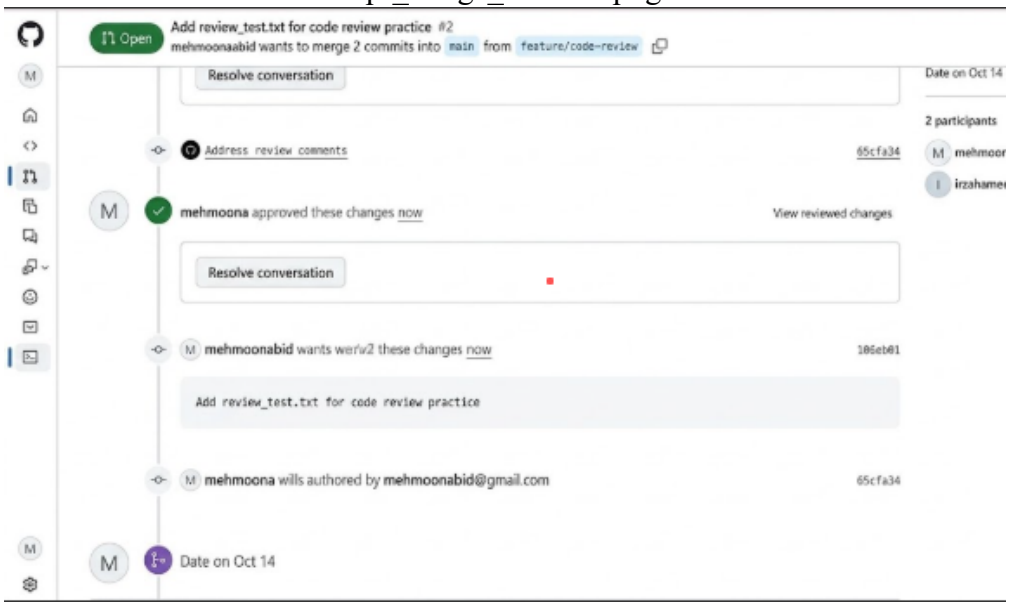
- Fix issues in the same branch and push new commits.
- PR updates automatically.

  Screenshot updated PR with new commits → pr_updated_with_commits.png

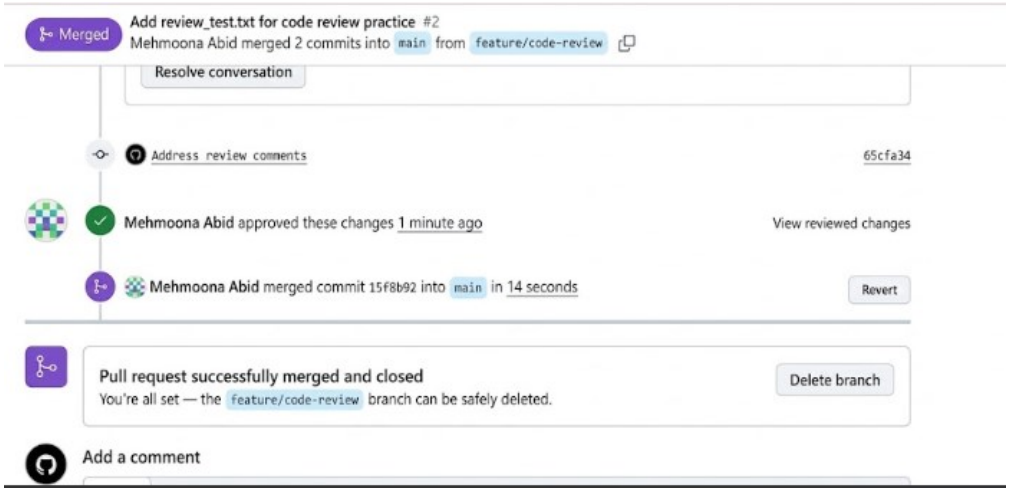


5. Merge the PR:

- Reviewer approves → merge using GitHub/GitLab UI.
- Choose merge style (merge commit, squash, or rebase).
- Screenshot confirmation → pr_merge_confirm.png.

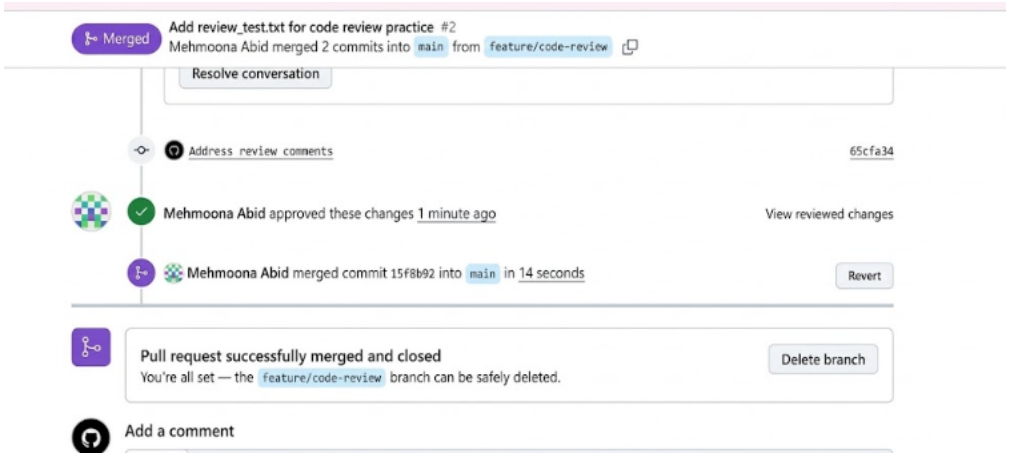


Screenshot result → pr_merged.png



6. Branch Handling After Merge:

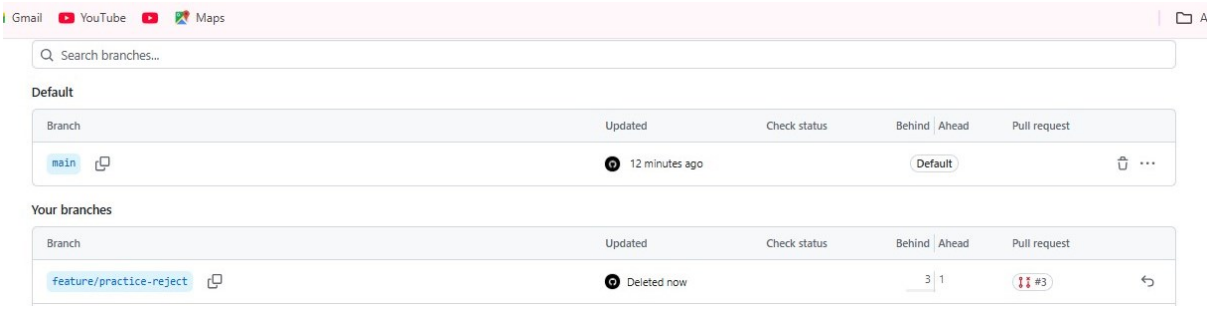
• Delete the merged PR branch and save the evidence as pr_branch_deleted.png.



Task 13: Branch Cleanup Best Practices

1. Delete Remote Branch After Merge:

- Delete a merged branch through the GitHub UI and save the evidence as remote_branch_deleted.png.



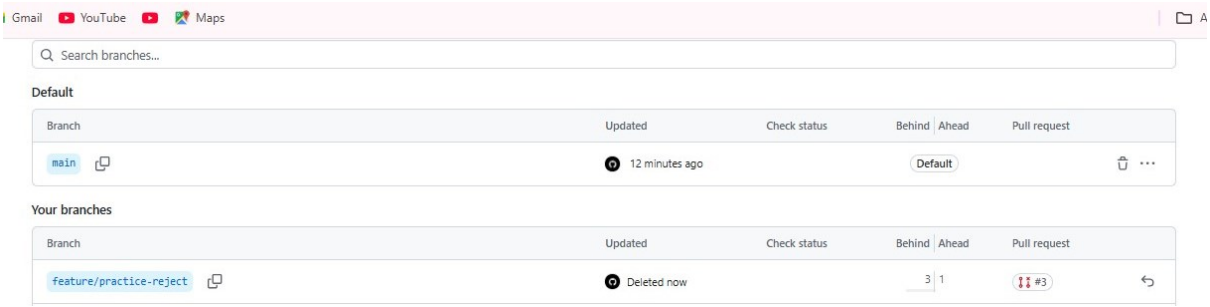
- Delete another completed remote branch from the terminal: *git push origin --delete <branch-name>*
- Save the terminal evidence as remote_branch_delete_cmd.png.

```
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git branch -r
  origin/HEAD -> origin/main
  origin/bugfix/user-auth-error
  origin/develop
  origin/feature-1
  origin/main
  origin/staging
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git push origin --delete feature-1
To github.com:mehmoonaabid/DevOps_Lab2_Practice.git
 - [deleted]         feature-1
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$ git branch -r
  origin/HEAD -> origin/main
  origin/bugfix/user-auth-error
  origin/develop
  origin/main
  origin/staging
mehmoonaabid@ubuntu:~/DevOps_Lab2_Practice$
```

2. Update Local Repository:

```
git checkout main
git pull origin main
git branch -d <branch-name>
git branch
```

3. Save the GitHub branch-deletion evidence as remote_branch_deleted.png.



4. Save the terminal command used to delete a remote branch as remote_branch_delete_cmd.png.



Exam Evaluation Questions

The three Q1_*.png files listed in the mandatory filename section are part of the current automated score. Q1_merge_history.png, Q2, and Q3 evidence are additional exam-practice evidence and may be reviewed separately.

Advanced Branching &

Merge Verification Create

a new branch in your

repository.

Screenshot as Q1_branch_created.png.

Make a small change in a file and commit it.

Screenshot as Q1_commit_done.png.

Merge this branch back into the main branch.

Screenshot as Q1_merge_done.png.

Show the history of commits in a way that verifies the merge.

Screenshot as Q1_merge_history.png.

Multi-Stage Workflow Simulation

Set up a branching workflow with three branches: main, develop, and staging.

Screenshot as Q2_branches_created.png.

Create a feature branch from develop, make changes, and commit them.

Screenshot as Q2_feature_branch_commit.png.

Merge the feature branch into develop.

Screenshot as Q2_merge_into_develop.png.

Merge develop into staging.

Screenshot as Q2_merge_into_staging.png.

Merge staging into main.

Screenshot as Q2_merge_into_main.png.

Provide proof that each stage contains the updated changes before it reaches main.

Screenshot as Q2_stage_verification.png.

Collaboration & Conflict Resolution

Work with a collaborator: both contributors should modify the same file but in separate branches.

Screenshots as Q3_branches_modified1.png, Q3_branches_modified2.png, and Q3_branches_modified3.png.

Attempt to merge the branches and capture the conflict.

Screenshot as Q3_merge_conflict.png.

Resolve the conflict so that both contributions are preserved.

Screenshot as Q3_conflict_resolved.png.

Provide evidence that the final version of the file contains both collaborators' changes.

Screenshot as Q3_final_file.png.

Submission

Before the deadline, verify the following:

Your repository is named exactly DevOps.

Your work is stored under Labs/Lab02.

Every required screenshot is stored under Labs/Lab02/screenshots.

Screenshot filenames match the required names exactly.

Every required screenshot clearly shows your standardized terminal identity, task evidence, and relevant output.

README.md this file.

No password, private key, token, or secret is visible.

All files are committed and pushed to GitHub.

Use these commands for the final submission check:

git commit -m

"Submit Lab 02

evidence" git push

origin main

Confirm on GitHub that the files are visible before considering the submission complete.

> This lab is designed for hands-on mastery of essential Git concepts as covered in your course materials. All steps reflect real workflows you'll use in professional software development.